

Cloud-Based Web System Design

A Privacy-Aware Cyber Resilience Assessment Platform for Small and Medium Organizations

Student	Taher Amine ELHOUARI
Program	15-Month Software Engineering Program
Checkpoint	Master Degree Highlights - Cloud-Based Web System Design
Project Type	Research-driven cloud web application
Date	19 August 2026

Executive Summary

CyberResilience Cloud is a cloud-native decision-support application designed to help small and medium organizations translate cybersecurity uncertainty into a prioritized improvement roadmap. The system addresses a recurring practical problem: smaller organizations often have limited security staff, fragmented visibility, and difficulty converting comprehensive cybersecurity frameworks into an actionable sequence of decisions. The project therefore focuses on a concise assessment workflow aligned to the six functions of the NIST Cybersecurity Framework (CSF) 2.0 - Govern, Identify, Protect, Detect, Respond, and Recover - and produces normalized domain scores, an overall maturity band, and prioritized recommendations.

The implementation uses a deliberately lightweight architecture: a responsive browser interface, a Node.js serverless scoring API, Vercel edge delivery, GitHub-based continuous integration, and an optional PostgreSQL persistence layer for authenticated production use. The public demonstrator follows a zero-retention pattern: anonymous responses are processed for scoring but are not stored by the server. This limits unnecessary data collection while still demonstrating a realistic cloud API boundary, input validation, security headers, testing, and deployment practices.

The design is research-informed. NIST SP 1300 specifically targets small-to-medium businesses with modest or no cybersecurity plans and provides a practical entry point for using CSF 2.0. CISA's Cross-Sector Cybersecurity Performance Goals similarly emphasize a limited set of high-impact actions that are intended to be practical for smaller organizations. OWASP ASVS 5.0 provides an application-security verification perspective for the web system itself. Together, these sources support the project's central design choice: prioritize clarity, measurability, safe defaults, and incremental improvement rather than presenting users with a large undifferentiated control catalogue.

1. Real-World Problem and Research Context

Cybersecurity guidance is abundant, but the ability to operationalize it is uneven. Small and medium organizations may depend heavily on cloud services, SaaS platforms, outsourced IT, remote work, and digital payment or collaboration systems while lacking a dedicated security architecture, governance team, or formal risk function. This creates a decision problem: leaders need to know which gaps matter most, which actions should be prioritized first, and how to communicate progress without requiring specialist knowledge of hundreds of controls.

NIST's Cybersecurity Framework 2.0 provides a broad outcomes-based structure that is intended to be usable across organization sizes, sectors, and maturity levels. NIST SP 1300 was published specifically as a Small Business Quick-Start Guide for small-to-medium businesses with modest or no cybersecurity plans. That source is directly relevant to this project because it validates the need for a practical entry point rather than a full-scale governance transformation on day one. The six CSF 2.0 functions also offer a useful systems-thinking model: governance shapes direction,

identification establishes understanding, protection reduces exposure, detection provides visibility, response limits harm, and recovery restores operations.

CISA's Cross-Sector Cybersecurity Performance Goals reinforce this prioritization approach. CISA describes the CPGs as a baseline set of high-impact cybersecurity practices and explicitly notes their usefulness for small- and medium-sized organizations. The CPGs are also organized around the NIST CSF functions. This supports the project's decision to generate focused improvement actions from the weakest domains instead of merely returning a score. A meaningful assessment should lead to decisions, ownership, and measurable progress.

The project therefore defines the core problem as follows: how can a cloud-based web system convert recognized cybersecurity outcomes into a fast, understandable, privacy-aware self-assessment that helps resource-constrained organizations prioritize improvements? The objective is not to certify compliance or replace a professional security assessment. It is to provide a structured starting point and a repeatable decision-support mechanism.

2. Requirements and Design Goals

The functional requirements were derived from the checkpoint objectives and the research context. The system must present a concise assessment, cover all six CSF 2.0 functions, validate user input, calculate normalized domain and overall scores, categorize maturity, identify weak areas, generate prioritized recommendations, and present results in a form that can be printed or saved as a PDF. The interface must work on desktop and mobile devices and should be understandable to non-specialists.

Non-functional requirements are equally important. The public demonstrator should minimize data collection, respond quickly, scale without server management, support automated testing and continuous integration, and expose a simple health endpoint. Security requirements include no-store responses for assessment results, browser security headers, strict input validation, limited HTTP methods, and avoidance of unnecessary secrets. The production architecture must also show how persistent organization and assessment history could be introduced using PostgreSQL without making persistence mandatory for anonymous use.

A central design goal is explainability. A user should understand why the platform asks the questions, what a score means, and what to do next. The maturity labels - Initial, Developing, Defined, and Adaptive - are deliberately descriptive rather than presented as a formal certification scale. The system makes clear that its role is prioritization and improvement planning.

3. System Architecture

The architecture uses four logical layers. First, the presentation layer is a responsive HTML, CSS, and JavaScript interface. It is served close to the user through Vercel's edge network and is intentionally framework-light for the demonstrator, reducing bundle complexity and startup

overhead. Second, the application layer contains the assessment workflow and result rendering. Third, the serverless API exposes scoring and health endpoints. The scoring function independently validates every domain code and answer value before calculation. Fourth, an optional PostgreSQL data layer supports authenticated production scenarios in which organizations explicitly choose to retain assessment history.

The system is intentionally stateless in public demo mode. This matters both technically and ethically. Serverless execution benefits from stateless request handling, and the zero-retention design means the system does not create a database of potentially sensitive descriptions of security weaknesses merely to demonstrate functionality. If persistence is enabled later, the included schema separates organizations, assessments, and domain scores, and indexes organization/time access patterns for historical reporting.

The cloud deployment model is Git-centric. Source code is maintained in GitHub, tests execute in GitHub Actions using Node's built-in test runner, and production delivery targets Vercel. Static assets are cached at the edge while the scoring function is invoked only when required. This architecture provides low operational overhead, horizontal scalability through the platform, and a clear separation between client code and trusted server-side scoring logic.

4. Technology Selection and Justification

The frontend uses standards-based HTML, CSS, and JavaScript because the primary interaction is a structured assessment rather than a complex multi-page application. This reduces dependency risk and demonstrates that architectural quality does not require unnecessary framework complexity. The interface uses semantic sections, form controls, keyboard-focus behavior, responsive breakpoints, and print styling. The scoring API uses Node.js because it aligns with the broader JavaScript learning pathway and can run efficiently in a serverless environment.

Vercel was selected for the target cloud platform because the project combines static delivery with small serverless functions. It provides a simple path from repository to public deployment, isolates API execution from the browser, and removes the need to maintain a persistent application server for the demonstrator. GitHub Actions supplies an independent CI gate so scoring logic is tested before changes are considered ready.

PostgreSQL was selected for the optional persistent model because retained assessments have clear relational structure: one organization can have many assessments, and each assessment has six domain results. A relational schema provides integrity constraints, simple historical queries, and straightforward indexing. The public demonstrator does not require a database connection, which keeps the deployment reproducible and avoids collecting unnecessary data.

5. Application Implementation

The implemented assessment contains eighteen questions, with three questions mapped to each CSF function. Responses use a five-point scale from 0 (not in place) to 4 (consistent and measured). For each domain, the arithmetic mean of the three responses is normalized to a 0-100 score. The overall score is the arithmetic mean of the six domain scores. This design gives each CSF function equal weight and makes the result easy to explain.

Maturity bands provide an additional interpretation layer. Scores below 40 are categorized as Initial, 40-59 as Developing, 60-79 as Defined, and 80-100 as Adaptive. The three lowest-scoring domains drive the recommendation set. This ensures that the result is not merely descriptive; it points the user toward the areas where improvement would have the greatest effect on balance across the resilience lifecycle.

The interface includes progressive flow from problem context to assessment, results, and architecture. Results are rendered as an overall score, domain bars, maturity text, and priority actions. A print action allows the user to save the result using the browser's PDF capability. The application also includes a local scoring fallback so the interface can still demonstrate behavior if the serverless endpoint is temporarily unavailable; the production API remains the preferred scoring path.

6. Security and Privacy Engineering

Security is part of the system design rather than an afterthought. The server-side scoring function rejects unsupported methods, verifies that the request contains a non-empty answer array, validates every domain identifier, converts values to numbers, and rejects values outside the 0-4 range. API responses use no-store caching instructions because assessment results may reveal security weaknesses. Browser headers include X-Content-Type-Options, a restrictive referrer policy, and a permissions policy that disables camera, microphone, and geolocation features that the application does not require.

The public deployment uses a zero-retention model. No account is required and the API does not persist assessment answers. This follows the principle of data minimization: if the demonstrator can achieve its educational purpose without storing sensitive security posture information, storage is unnecessary risk. The PostgreSQL schema is therefore positioned as an explicit production extension rather than silently enabled behavior.

OWASP ASVS 5.0 is used as an application-security reference for the development mindset. ASVS provides structured verification requirements spanning architecture, validation and business logic, frontend security, APIs, authentication, authorization, cryptography, secure communication, configuration, data protection, and logging. The current demonstrator intentionally avoids authentication and sensitive persistent data, which reduces attack surface, but a production version with accounts would require systematic verification against relevant ASVS requirements.

7. Cloud Deployment, CI/CD, and Scalability

The application is structured for cloud-native deployment. Static files can be distributed globally through the platform CDN, while the scoring endpoint runs as an on-demand function. This separates traffic growth from a single long-running server process. Stateless scoring is naturally horizontally scalable because any function instance can process any valid request without session affinity.

The repository includes a GitHub Actions workflow that executes the scoring tests on push and pull requests. The tests verify a complete six-domain assessment and invalid-value rejection. Automated testing is intentionally focused on the most consequential business logic in the current version. A production expansion would add browser-level tests, accessibility checks, API contract tests, dependency scanning, and performance tests.

The persistence model can scale independently. If authenticated history is introduced, managed PostgreSQL can be used with connection pooling and appropriate retention policies. The schema already includes an index for organization and creation time, supporting efficient retrieval of assessment history. Because persistence is not on the synchronous path of the public demo, the current system avoids database cold-start and connection-management complexity.

8. Evaluation and Results

The project was evaluated against the checkpoint's professional system criteria rather than only visual completion. Functionally, the application presents the full assessment, validates completion, calculates results, renders recommendations, supports printable output, and exposes a health endpoint. The scoring module was tested using Node's built-in test runner, with successful tests for a complete maximum-score assessment and invalid answer rejection.

Architecturally, the design demonstrates frontend/backend separation, API boundaries, an optional database model, cloud deployment configuration, continuous integration, and security headers. The application is also intentionally resilient to API unavailability through a local fallback, although this fallback is primarily a demonstration aid and production governance should decide whether local calculation is acceptable for the target use case.

From a UX perspective, the system minimizes terminology overload, keeps the response scale consistent across all questions, groups interpretation around familiar resilience functions, and converts the weakest domains into concrete actions. The responsive layout collapses multi-column sections for smaller screens. The use of text labels in addition to visual bars avoids relying only on color to communicate scores.

9. Limitations and Future Development

The current assessment is deliberately concise. Eighteen questions cannot provide the assurance of a formal audit, penetration test, regulatory review, or detailed control assessment. The equal weighting of the six functions is transparent and easy to understand, but a future version could allow sector-specific weighting, applicability logic, evidence uploads, confidence ratings, and organization profiles. Recommendation logic could also map to more granular CSF outcomes and CISA CPG actions.

A production platform could add authenticated multi-tenant workspaces, historical trend analysis, evidence management, role-based access control, exportable executive reports, and collaboration between business and security owners. PostgreSQL persistence would then be enabled with explicit consent, encryption and retention controls. A further research direction would compare whether organizations improve implementation rates more effectively when recommendations are presented as prioritized business outcomes rather than as control lists.

Cloud resilience can also be extended. Future versions could add synthetic availability monitoring, observability dashboards, rate limiting at the platform edge, distributed tracing, infrastructure-as-code, automated security scanning, and a disaster-recovery plan for the persistent mode.

10. Conclusion

CyberResilience Cloud demonstrates how research, software design, cloud architecture, security engineering, and user-centered decision support can be combined in a focused web system. The project begins with a real problem - the difficulty smaller organizations face in prioritizing cybersecurity improvement - and uses NIST CSF 2.0 and CISA guidance to shape a concise assessment model. The application then translates that model into a scalable serverless implementation with automated testing, privacy-aware defaults, and an optional relational persistence layer.

The most important design decision is that the application does not treat maturity scoring as the end product. The useful output is the prioritized roadmap that follows. By identifying weak domains and turning them into ownership-oriented actions, the system supports the practical purpose of cybersecurity governance: making better risk decisions and improving resilience over time.

Architecture Summary

Layer	Technology	Responsibility	Scaling Model
Presentation	HTML/CSS/JavaScript	Assessment UX, results, print output	CDN / edge cache
Application	Browser + Node.js	Workflow and scoring orchestration	Stateless

API	Vercel Serverless Functions	Validation, scoring, health	On-demand horizontal
Data	PostgreSQL (optional)	Organizations and assessment history	Managed relational database
CI/CD	GitHub Actions + Vercel	Automated test and deployment flow	Repository-driven

References

National Institute of Standards and Technology (NIST). (2024). NIST Cybersecurity Framework 2.0: Small Business Quick-Start Guide (NIST SP 1300). <https://doi.org/10.6028/NIST.SP.1300>

National Institute of Standards and Technology (NIST). Cybersecurity Framework 2.0 Quick Start Guides. <https://www.nist.gov/cyberframework/quick-start-guides>

Cybersecurity and Infrastructure Security Agency (CISA). Cross-Sector Cybersecurity Performance Goals. <https://www.cisa.gov/cross-sector-cybersecurity-performance-goals>

Cybersecurity and Infrastructure Security Agency (CISA). Cybersecurity Performance Goals: Frequently Asked Questions. <https://www.cisa.gov/cross-sector-cybersecurity-performance-goals/frequently-asked-questions>

OWASP Foundation. Application Security Verification Standard (ASVS) 5.0.0. <https://owasp.org/www-project-application-security-verification-standard/>

Appendix A - Scoring Model

Each question is answered on a 0-4 scale. For each CSF function, the mean of its three question scores is divided by 4 and multiplied by 100. The overall score is the arithmetic mean of the six normalized function scores. The three lowest-scoring functions are selected for prioritized recommendations.

Appendix B - Repository Deliverables

- Cloud web application source code
- Serverless scoring and health APIs
- PostgreSQL production schema
- GitHub Actions CI workflow
- Automated scoring tests
- Architecture documentation
- Professional report
- 10-minute presentation/demo script

Research-to-Design Traceability

The final design can be traced directly to the research evidence. NIST SP 1300 supports a simplified entry point for small and medium organizations, which led to the concise assessment format and plain-language questions. The six-function CSF 2.0 structure became the domain model used for both the user experience and the scoring engine. CISA's emphasis on measurable, high-impact actions influenced the decision to rank weak domains and generate prioritized next steps rather than present an undifferentiated list of controls. OWASP ASVS influenced the technical treatment of the web application, including explicit input validation, API boundary checks, restrictive methods, security-related HTTP headers, and minimization of sensitive persistence. This traceability is important because it shows that technology choices were not made independently of the problem statement. The cloud architecture, privacy model, scoring model, and result presentation are all consequences of the research question: how to make recognized cybersecurity outcomes practical, measurable, and usable for organizations with limited specialist resources.